

LAB | API and Integration Patterns Lab

Week 2

Day 4

-

API and Integration Patterns Fundamentals

Integration Patterns

API Integration (Notebooks)

LAB | API and Integration Patterns Lab

API and Integration Patterns Quiz

LAB | API and Integration Patterns Lab

LAB

*Mandatory

Lab M2.03 - Multi- Provider News Summarizer Scenario

You're a developer at a news analytics company. Your team needs to process hundreds of news articles daily

to provide summaries and sentiment analysis for clients. The system must be reliable (with fallback providers), efficient (processing multiple articles concurrently), and cost-effective (tracking API costs).

Your manager has asked you to build a production-ready news summarizer that:

- Fetches news articles from an external API
- Uses OpenAI to generate summaries (fast and cheap)
- Uses Anthropic Claude to analyze sentiment (better at nuance)
- Has fallback logic (if OpenAI fails, Anthropic does both)
- Tracks costs to stay within budget
- Handles rate limits properly

Real-world impact: This system will process 500+ articles per day, saving your team 20+ hours of manual work while providing consistent, high-quality analysis.

Note on LLM Providers: This lab uses OpenAI and Anthropic Claude as examples, but these are suggestions. You can use another LLM provider that works best for your to integrate with OpenAI. For a free alternative to test LLMs without payment, consider [Cohere](#) (deployment keys are still paid)

Learning Objectives

After completing this lab, you will be able to:

- Fetch data from external APIs and handle errors gracefully
- Use multiple LLM providers (OpenAI + Anthropic) in one application
- Pass outputs from one LLM to another LLM
- Implement fallback logic when a provider fails

- Apply rate limiting to avoid API quota violations
- Track API costs across multiple providers
- Write unit tests for API integration code
- Use environment variables for production configuration
- (Optional - Advanced) Process multiple items concurrently with `async/await`

Prerequisites

Before starting this lab, make sure you have:

- Completed Lessons 1 and 2 of this submodule
- Python 3.10+ installed
- OpenAI API key (sign up at platform.openai.com)
- Anthropic API key (sign up at console.anthropic.com)
- NewsAPI key (sign up for free at newsapi.org)

Estimated Time: 2-3 hours

Part 1: Setup and Environment Configuration

Step 1: Create Project Structure

Create the following folder structure:

Copy

news-summarizer/
├── .env
├── .gitignore
├── requirements.txt
├── config.py
├── news_api.py
├── llm_providers.py
└── summarizer.py

```
└─ test_summarizer.py
└─ main.py
```

[Explain this code](#)

Step 2: Install Dependencies

Create `requirements.txt`:

[Copy](#)

```
openai>=1.12.0
anthropic>=0.18.0
requests>=2.31.0
python-dotenv>=1.0.0
aiohttp>=3.9.0
tiktoken>=0.5.0
pytest>=7.4.0
```

[Explain this code](#)

Install dependencies:

[Copy](#)

```
pip install -r requirements.txt
```

[Explain this code](#)

Step 3: Configure Environment Variables

Create `.env` file (DO NOT commit this!):

[Copy](#)

```
# .env
OPENAI_API_KEY=your-openai-key-here
ANTHROPIC_API_KEY=your-anthropic-key-here
NEWS_API_KEY=your-newsapi-key-here

ENVIRONMENT=development
MAX_RETRIES=3
REQUEST_TIMEOUT=30
```

```
DAILY_BUDGET=5.00
```

Explain this code

Create `.gitignore`:

Copy

```
.env
.env.*
__pycache__/
*.pyc
*.pyo
*.log
.pytest_cache/
```

Explain this code

Step 4: Create Configuration Module

Create `config.py`:

Copy

```
"""Configuration management for news"""
import os
from dotenv import load_dotenv

# Load environment variables
load_dotenv()

class Config:
    """Application configuration."""

    # API Keys
    OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")
    ANTHROPIC_API_KEY = os.getenv("ANTHROPIC_API_KEY")
    NEWS_API_KEY = os.getenv("NEWS_API_KEY")

    # Environment
```

```

ENVIRONMENT = os.getenv("ENVIRONMENT",
# API Configuration
MAX_RETRIES = int(os.getenv("MAX_RETRIES", "3"))
REQUEST_TIMEOUT = int(os.getenv("REQUEST_TIMEOUT",
# Models
OPENAI_MODEL = "gpt-4o-mini"
ANTHROPIC_MODEL = "claude-3-5-sonnet-20241022"
# Cost Control
DAILY_BUDGET = float(os.getenv("DAILY_BUDGET",
# Rate Limits (requests per minute)
OPENAI_RPM = 500
ANTHROPIC_RPM = 50
NEWS_API_RPM = 100
@classmethod
def validate(cls):
    """Validate that required configuration is
    required = [
        ("OPENAI_API_KEY", cls.OPENAI_API_KEY),
        ("ANTHROPIC_API_KEY", cls.ANTHROPIC_API_KEY),
        ("NEWS_API_KEY", cls.NEWS_API_KEY)
    ]
    missing = [name for name, value in required if not
    if missing:
        raise ValueError(f"Missing required configuration:
    print(f"✓ Configuration validated for
# Validate on import
Config.validate()

```

Explain this code

✓ **Checkpoint 1:** Run `python config.py` - it should validate successfully (or tell you which keys are missing).

Part 2: News API Integration

Step 5: Create News API Module

Create `news_api.py`:

Copy

```
"""News API integration module."""
import requests
import time
from config import Config

class NewsAPI:
    """Fetch news articles from NewsAPI."""

    def __init__(self):
        self.api_key = Config.NEWS_API_KEY
        self.base_url = "https://newsapi.org/v2"
        self.last_call_time = 0
        self.min_interval = 60.0 / Config.NEWS_API_RPM #

    def _wait_if_needed(self):
        """Wait if we need to rate limit."""
        elapsed = time.time() - self.last_call_time
        if elapsed < self.min_interval:
            wait_time = self.min_interval - elapsed
            print(f"Rate limiting News API: waiting {wait_time} seconds")
            time.sleep(wait_time)
            self.last_call_time = time.time()

    def fetch_top_headlines(self,
```

```
"""
```

```
Fetch top headlines.
```

```
Args:
```

```
category: News category (business, technology,
```

```
country: Country code (us, gb, etc.)
```

```
max_articles: Maximum number of articles to return
```

```
Returns:
```

```
List of article dictionaries
```

```
"""
```

```
self._wait_if_needed()
```

```
url = f"{self.base_url}/top-headlines"
```

```
params = {
```

```
"apiKey": self.api_key,
```

```
"category": category,
```

```
"country": country,
```

```
"pageSize": max_articles
```

```
}
```

```
try:
```

```
response = requests.get(url, params=params,
```

```
response.raise_for_status()
```

```
data = response.json()
```

```
if data.get("status") != "ok":
```

```
raise Exception(f"News API error: {data.get('status')}")
```

```
articles = data.get("articles", [])
```

```
# Extract relevant fields
```

```
processed_articles = []
```

```
for article in articles:
```

```
processed_articles.append({
```

```
"title": article.get("title", ""),
```

```

        "description": article.get("description", ""),
        "content": article.get("content", ""),
        "url": article.get("url", ""),
        "source": article.get("source", {}).get("name",
        "published_at": article.get("publishedAt", "")
    })

    print(f"✓ Fetched {len(processed_articles)}
    return processed_articles

except requests.exceptions.RequestException as e:
    print(f"✗ Error fetching news: {e}")
    return []

# Test the module
if __name__ == "__main__":
    api = NewsAPI()
    articles =

    for i, article in enumerate(articles, 1):
        print(f"\n{i}. {article['title']}")
        print(f"    Source: {article['source']}")
        print(f"    URL: {article['url']}")

```

Explain this code

✓ **Checkpoint 2:** Run `python news_api.py` - it should fetch and display 3 tech news articles.

Part 3: LLM Provider

Integration

Step 6: Create LLM Providers Module

Create `llm_providers.py`:

Copy

```
"""LLM provider integration with fallback"""
import os
import time
import tiktoken
from openai import OpenAI
from anthropic import Anthropic
from config import Config

# Pricing (per million tokens)
PRICING = {
    "gpt-4o-mini": {"input": 0.15, "output": 0.60},
    "gpt-4o": {"input": 2.50, "output": 10.00},
    "claude-3-5-sonnet-20241022": {"input": 3.00,
    "output": 15.00}
}


class CostTracker:
    """Track API costs."""

    def __init__(self):
        self.total_cost = 0.0
        self.requests = []

    def track_request(self, provider, model,
        """Track a single request."""
        pricing = PRICING.get(model, {"input": 3.0,
        "output": 15.0})
        input_cost = (input_tokens / 1_000_000) *
        output_cost = (output_tokens / 1_000_000) *
        cost = input_cost + output_cost

        self.total_cost += cost
        self.requests.append({
            "provider": provider,
            "model": model,
            "input_tokens": input_tokens,
```

```

        "output_tokens": output_tokens,
        "cost": cost
    })

    return cost

def get_summary(self):
    """Get cost summary."""
    total_input = sum(r["input_tokens"] for r in self.requests)
    total_output = sum(r["output_tokens"] for r in self.requests)

    return {
        "total_requests": len(self.requests),
        "total_cost": self.total_cost,
        "total_input_tokens": total_input,
        "total_output_tokens": total_output,
        "average_cost": self.total_cost / len(self.requests)
    }

def check_budget(self, daily_budget):
    """Check if we're within budget."""
    if self.total_cost >= daily_budget:
        raise Exception(f"Daily budget of $ {daily_budget} exceeded. Current cost: $ {self.total_cost}")

    percent_used = (self.total_cost / daily_budget) * 100
    if percent_used >= 90:
        print(f"⚠ Warning: {percent_used:.1f}% of daily budget used")

def count_tokens(text, model="gpt-4o-mini"):
    """Count tokens in text."""
    try:
        encoding = tiktoken.encoding_for_model(model)
        return len(encoding.encode(text))
    except:
        # Fallback: estimate 4 characters per token
        return len(text) // 4

```

```
class LLMProviders:
```

```
    """Manage multiple LLM providers with fallback."""
```

```
    def __init__(self):
```

```
        self.openai_client =
```

```
        self.anthropic_client =
```

```
        self.cost_tracker = CostTracker()
```

```
    # Rate limiting
```

```
    self.openai_last_call = 0
```

```
    self.anthropic_last_call = 0
```

```
    self.openai_interval = 60.0 / Config.OPENAI_RPM
```

```
    self.anthropic_interval = 60.0 /
```

```
    def _wait_openai(self):
```

```
        """Rate limit OpenAI calls."""
```

```
        elapsed = time.time() - self.openai_last_call
```

```
        if elapsed < self.openai_interval:
```

```
            time.sleep(self.openai_interval - elapsed)
```

```
            self.openai_last_call = time.time()
```

```
    def _wait_anthropic(self):
```

```
        """Rate limit Anthropic calls."""
```

```
        elapsed = time.time() - self.anthropic_last_call
```

```
        if elapsed < self.anthropic_interval:
```

```
            time.sleep(self.anthropic_interval - elapsed)
```

```
            self.anthropic_last_call = time.time()
```

```
    def ask_openai(self, prompt, model=None):
```

```
        """Ask OpenAI."""
```

```
        if model is None:
```

```
            model = Config.OPENAI_MODEL
```

```
        self._wait_openai()
```

```
input_tokens = count_tokens(prompt, model)
```

```
response =  
model=model,  
messages=[{"role": "user", "content": prompt}]  
)
```

```
output_text = response.choices[0].message.content  
output_tokens = count_tokens(output_text, model)
```

```
# Track cost
```

```
cost = self.cost_tracker.track_request("openai",  
self.cost_tracker.check_budget(Config.DAILY_BUDGET
```

```
return output_text
```

```
def ask_anthropic(self, prompt, model=None):
```

```
    """Ask Anthropic."""
```

```
    if model is None:
```

```
        model = Config.ANTHROPIC_MODEL
```

```
    self._wait_anthropic()
```

```
input_tokens = count_tokens(prompt, model)
```

```
response = self.anthropic_client.messages.create(  
model=model,  
max_tokens=1024,  
messages=[{"role": "user", "content": prompt}]  
)
```

```
output_text = response.content[0].text  
output_tokens = count_tokens(output_text, model)
```

```
# Track cost
```

```
cost =
```

```

self.cost_tracker.check_budget(Config.DAILY_BUDGET)

return output_text

def ask_with_fallback(self, prompt,
    """
    Ask with fallback to secondary provider.

    Args:
    prompt: Question to ask
    primary: Primary provider ("openai" or
    "anthropic")

    Returns:
    Dictionary with provider used and response
    """
    try:
        if primary == "openai":
            print("Trying OpenAI (primary)...")
            response = self.ask_openai(prompt)
            return {"provider": "openai", "response":
        else:
            print("Trying Anthropic (primary)...")
            response = self.ask_anthropic(prompt)
            return {"provider": "anthropic", "response":

    except Exception as e:
        print(f"x Primary provider failed: {e}")
        print("Falling back to secondary provider...")

    try:
        if primary == "openai":
            response = self.ask_anthropic(prompt)
            return {"provider": "anthropic", "response":
        else:
            response = self.ask_openai(prompt)
            return {"provider": "openai", "response":

```

```

except Exception as e2:
    print(f"x Secondary provider also failed: {e2}")
    raise Exception("All providers failed")

# Test the module
if __name__ == "__main__":
    providers = LLMPProviders()

# Test OpenAI
print("Testing OpenAI:")
response = providers.ask_openai("What is Python?")
print(f"Response: {response}\n")

# Test Anthropic
print("Testing Anthropic:")
response = providers.ask_anthropic("What is")
print(f"Response: {response}\n")

# Test fallback
print("Testing fallback:")
result = providers.ask_with_fallback("What is")
print(f"Provider used: {result['provider']}")
print(f"Response: {result['response']}\n")

# Show cost summary
summary = providers.cost_tracker.get_summary()
print(f"Total cost: ${summary['total_cost']:.4f}")
print(f"Total requests: ")

```

Explain this code

✓ **Checkpoint 3:** Run `python llm_providers.py` - it should test both providers and show cost tracking.

Part 4: News Summarizer

Core Logic

Step 7: Create Summarizer Module

Create `summarizer.py`:

Copy

```
"""News summarizer with multi-provider support."""
import asyncio
import aiohttp
from news_api import NewsAPI
from llm_providers import LLMPProviders


class NewsSummarizer:
    """Summarize news articles using multiple LLM
    providers"""

    def __init__(self):
        self.news_api = NewsAPI()
        self.llm_providers = LLMPProviders()

    def summarize_article(self, article):
        """
        Summarize a single article.

        Args:
            article: Article dictionary

        Returns:
            Dictionary with summary and sentiment
        """
        print(f"\nProcessing: {article['title'][:60]}...")

        # Prepare text for summarization
        article_text = f"""Title: {article['title']}
        Description: {article['description']}
```

```
Content: {article['content'][:500]}""" # Limit
```

```
# Step 1: Summarize with OpenAI (fast and cheap)
```

```
try:
```

```
print(" → Summarizing with OpenAI...")
```

```
summary_prompt = f"""Summarize this news article
```

```
{article_text}"""
```

```
summary =
```

```
print(f" ✓ Summary generated")
```

```
except Exception as e:
```

```
print(f" ✗ OpenAI summarization failed: {e}")
```

```
# Fallback to Anthropic for summary
```

```
print(" → Falling back to Anthropic for
```

```
summary =
```

```
# Step 2: Analyze sentiment with Anthropic (better
```

```
# Note: Using Anthropic for sentiment analysis is
```

```
# that works best for your needs. For a free
```

```
# consider Cohere: https://dashboard.cohere.com/
```

```
try:
```

```
print(" → Analyzing sentiment with Anthropic...")
```

```
sentiment_prompt = f"""Analyze the sentiment of
```

```
Provide:
```

```
– Overall sentiment (positive/negative/neutral)
```

```
– Confidence (0–100%)
```

```
– Key emotional tone
```

```
Be concise (2–3 sentences)."""
```

```
sentiment =
```

```
print(f" ✓ Sentiment analyzed")
```

```
except Exception as e:
    print(f" x Anthropic sentiment analysis failed: {e}")
    # If sentiment fails, use a fallback
    sentiment = "Unable to analyze sentiment"
```

```
return {
    "title": article['title'],
    "source": article['source'],
    "url": article['url'],
    "summary": summary,
    "sentiment": sentiment,
    "published_at": article['published_at']
}
```

```
def process_articles(self, articles):
```

```
    """
```

```
    Process multiple articles.
```

```
    Args:
```

```
    articles: List of article dictionaries
```

```
    Returns:
```

```
    List of processed articles
```

```
    """
```

```
    results = []
```

```
    for article in articles:
```

```
        try:
```

```
            result = self.summarize_article(article)
```

```
            results.append(result)
```

```
        except Exception as e:
```

```
            print(f"x Failed to process article: {e}")
```

```
            # Continue with next article
```

```
    return results
```

```

def generate_report(self, results):
    """Generate a summary report."""
    print("\n" + "="*80)
    print("NEWS SUMMARY REPORT")
    print("="*80)

    for i, result in enumerate(results, 1):
        print(f"\n{i}. {result['title']}")
        print(f"    Source: {result['source']} | Published: {result['published']}")
        print(f"    URL: {result['url']}")
        print(f"\n    SUMMARY:")
        print(f"    {result['summary']}")
        print(f"\n    SENTIMENT:")
        print(f"    {result['sentiment']}")
        print(f"\n    {'-'*76}")

    # Cost summary
    summary = self.get_summary()
    print("\n" + "="*80)
    print("COST SUMMARY")
    print("="*80)
    print(f"Total requests: {summary['total_requests']}")
    print(f"Total cost: ${summary['total_cost']:.4f}")
    print(f"Total tokens: {summary['total_tokens']}")
    print(f"    Input: {summary['input_tokens']}")
    print(f"    Output: {summary['output_tokens']}")
    print(f"Average cost per request: ${summary['avg_cost_per_request']:.4f}")
    print("="*80)

    # Test the module
    if __name__ == "__main__":
        summarizer = NewsSummarizer()

    # Fetch news
    print("Fetching news articles...")
    articles = self.fetch_news()

```

```

if not articles:
    print("No articles fetched. Check your News API")
else:
    # Process articles
    print(f"\nProcessing {len(articles)} articles...")
    results = summarizer.process_articles(articles)

    # Generate report
    summarizer.generate_report(results)

```

Explain this code

✓ **Checkpoint 4:** Run `python summarizer.py` - it should fetch 2 articles, summarize them with OpenAI, analyze sentiment with Anthropic, and show a cost summary.

Part 5: Async Processing (Optional Advanced Feature)

Step 8: Add Async Support for Concurrent Processing (Optional - Advanced)

Add this to `summarizer.py` (below the `NewsSummarizer` class):

Copy

```

class AsyncNewsSummarizer(NewsSummarizer):
    """Async version for processing multiple articles"""

    async def summarize_article_async(self, article):
        """Async version of summarize_article."""
        # Note: The LLM API calls themselves are not async

```

```
# For true async, you'd need to use aiohttp with  
# This version just allows concurrent processing
```

```
return await
```

```
async def process_articles_async(self, articles,  
    """
```

```
    Process articles concurrently.
```

```
    Args:
```

```
    articles: List of articles
```

```
    max_concurrent: Maximum concurrent processes
```

```
    Returns:
```

```
    List of results
```

```
    """
```

```
    semaphore = asyncio.Semaphore(max_concurrent)
```

```
async def process_with_semaphore(article):
```

```
async with semaphore:
```

```
return await self.summarize_article_async(article)
```

```
    tasks = [process_with_semaphore(article) for  
    results = await asyncio.gather(*tasks,
```

```
# Filter out exceptions
```

```
    valid_results = [r for r in results if not
```

```
return valid_results
```

```
# Test async version
```

```
async def test_async():
```

```
    summarizer = AsyncNewsSummarizer()
```

```
# Fetch more articles
```

```
    print("Fetching news articles...")
```

```

articles =

if articles:
    print(f"\nProcessing {len(articles)} articles")
    results = await
    summarizer.generate_report(results)

if __name__ == "__main__":
    # Uncomment to test async version
    # asyncio.run(test_async())
    pass

```

Explain this code

Part 6: Testing

Step 9: Write Unit Tests

Create `test_summarizer.py`:

Copy

```

"""Unit tests for news summarizer."""
import pytest
from unittest.mock import Mock, patch
from news_api import NewsAPI
from llm_providers import LLMProviders,
from summarizer import NewsSummarizer

class TestCostTracker:
    """Test cost tracking functionality."""

    def test_track_request(self):
        """Test tracking a single request."""
        tracker = CostTracker()
        cost = tracker.track_request("openai", "gpt-4o-

assert cost > 0

```

```

assert tracker.total_cost == cost
assert len(tracker.requests) == 1

def test_get_summary(self):
    """Test summary generation."""
    tracker = CostTracker()
    tracker.track_request("openai", "gpt-4o-mini",
    tracker.track_request("anthropic", "claude-3-5-

summary = tracker.get_summary()

assert summary["total_requests"] == 2
assert summary["total_cost"] > 0
assert summary["total_input_tokens"] == 250
assert summary["total_output_tokens"] == 500

def test_budget_check(self):
    """Test budget checking."""
    tracker = CostTracker()

    # Should not raise for small amount
    tracker.track_request("openai", "gpt-4o-mini",
    tracker.check_budget(10.00) # Should pass

    # Should raise for exceeding budget
    tracker.total_cost = 15.00
    with pytest.raises(Exception,
    tracker.check_budget(10.00)

class TestTokenCounting:
    """Test token counting."""

    def test_count_tokens(self):
        """Test token counting function."""
        text = "Hello, how are you?"
        count = count_tokens(text)

```

```
assert count > 0
assert count < len(text)  # Should be less than
```

```
class TestNewsAPI:
    """Test News API integration."""
```

```
@patch('news_api.requests.get')
def test_fetch_top_headlines(self, mock_get):
    """Test fetching headlines."""
    # Mock successful response
    mock_response = Mock()
    mock_response.status_code = 200
    mock_response.json.return_value = {
        "status": "ok",
        "articles": [
            {
                "title": "Test Article",
                "description": "Test description",
                "content": "Test content",
                "url": "https://example.com",
                "source": {"name": "Test Source"},
                "publishedAt": "2026-01-19"
            }
        ]
    }
    mock_get.return_value = mock_response
```

```
api = NewsAPI()
articles = api.fetch_top_headlines(max_articles=1)
```

```
assert len(articles) == 1
assert articles[0]["title"] == "Test Article"
assert articles[0]["source"] == "Test Source"
```

```
class TestLLMProviders:
```

```

"""Test LLM provider integration."""

@patch('llm_providers.OpenAI')
def test_ask_openai(self, mock_openai_class):
    """Test OpenAI integration."""
    # Mock OpenAI client
    mock_client = Mock()
    mock_response = Mock()
    mock_response.choices = [{"text": "Test response"}]
    mock_client.chat.completions.create.return_value = mock_response
    mock_openai_class.return_value = mock_client

    providers = LLMPProviders()
    providers.openai_client = mock_client

    response = providers.ask_openai("Test prompt")

    assert response == "Test response"
    assert mock_client.chat.completions.create.called

class TestNewsSummarizer:
    """Test news summarizer."""

    def test_initialization(self):
        """Test summarizer initialization."""
        summarizer = NewsSummarizer()

        assert summarizer.news_api is not None
        assert summarizer.llm_providers is not None

    @patch.object(LLMPProviders, 'ask_openai')
    @patch.object(LLMPProviders, 'ask_anthropic')
    def test_summarize_article(self, mock_anthropic, mock_openai):
        """Test article summarization."""
        mock_openai.return_value = "Test summary"
        mock_anthropic.return_value = "Positive sentiment"

```

```

summarizer = NewsSummarizer()
article = {
    "title": "Test Article",
    "description": "Test description",
    "content": "Test content",
    "url": "https://example.com",
    "source": "Test Source",
    "published_at": "2026-01-19"
}

result = summarizer.summarize_article(article)

assert result["title"] == "Test Article"
assert result["summary"] == "Test summary"
assert result["sentiment"] == "Positive sentiment"
assert mock_openai.called
assert mock_anthropic.called

# Run tests
if __name__ == "__main__":
    pytest.main([__file__, "-v"])

```

Explain this code

✓ **Checkpoint 5:** Run pytest

`test_summarizer.py -v` - all tests should pass.

Part 7: Main Application

Step 10: Create Main Application

Create `main.py`:

Copy

```

"""Main application entry point."""
import sys
from summarizer import NewsSummarizer,

```

```
import asyncio
```

```
def main():
```

```
    """Run the news summarizer."""
```

```
    print("="*80)
```

```
    print("NEWS SUMMARIZER – Multi-Provider Edition")
```

```
    print("="*80)
```

```
    # Get user input
```

```
    category = input("\nEnter news category  
(technology/business/health/general): ").strip()
```

```
    num_articles = input("How many articles to
```

```
    try:
```

```
        num_articles = int(num_articles)
```

```
        num_articles = max(1, min(10, num_articles)) #
```

```
    except:
```

```
        num_articles = 3
```

```
    use_async = input("Use async processing? (y/n):
```

```
    print(f"\nFetching {num_articles} articles from
```

```
    try:
```

```
        if use_async:
```

```
            # Use async version
```

```
            summarizer = AsyncNewsSummarizer()
```

```
            articles =
```

```
            category=category,
```

```
            max_articles=num_articles
```

```
        )
```

```
    if articles:
```

```
        print(f"\nProcessing {len(articles)} articles
```

```
        results = asyncio.run(
```

```
        summarizer.process_articles_async(articles,
```

```

)
summarizer.generate_report(results)

else:
    # Use synchronous version
    summarizer = NewsSummarizer()
    articles = fetch_articles(category=category,
                              max_articles=num_articles)

    if articles:
        print(f"\nProcessing {len(articles)} articles...")
        results = summarizer.process_articles(articles)
        summarizer.generate_report(results)

    print("\n✓ Processing complete!")

except KeyboardInterrupt:
    print("\n\nOperation cancelled by user.")
    sys.exit(0)

except Exception as e:
    print(f"\n✗ Error: {e}")
    sys.exit(1)

if __name__ == "__main__":
    main()

```

Explain this code

✓ **Checkpoint 6:** Run `python main.py` - the interactive application should fetch and process news articles.

Part 8: Deliverables and Submission

What to Submit

Create a GitHub repository with the following:

1. All source code files:

- `config.py`
- `news_api.py`
- `llm_providers.py`
- `summarizer.py`
- `test_summarizer.py`
- `main.py`

2. Configuration files:

- `requirements.txt`
- `.gitignore`
- `.env.example` (template without real keys)

3. Documentation:

- `README.md` explaining:
 - What the project does
 - How to set it up
 - How to run it
 - Example output
 - Cost analysis

4. Test results:

- Screenshot or output of `pytest` passing all tests
- Screenshot of main app processing 3-5 articles

Submission Guidelines

1. Code Quality Checklist:

- All code is well-commented
- Functions have docstrings
- No API keys committed to git
- `.env` is in `.gitignore`
- All tests pass
- Code follows PEP 8 style guide

2. Functionality Checklist:

- Fetches news from external API
- Uses OpenAI for summarization
- Uses Anthropic for sentiment analysis
- Implements fallback logic
- Tracks API costs
- Respects rate limits
- Has working unit tests

3. Submit:

- Repository URL
- Brief write-up (200-300 words) explaining:
 - Challenges you faced
 - How you solved them
 - What you learned
 - Ideas for improvement

Bonus Challenges (Optional)

Want to go further? Try these:

1. **Caching:** Add response caching to avoid re-processing identical articles
2. **Database:** Store processed articles in SQLite database
3. **Web UI:** Create a simple Flask/FastAPI web interface
4. **Scheduling:** Add ability to run automatically on a schedule
5. **Email Reports:** Send daily email summaries
6. **More Providers:** Add Google AI (Gemini) as a third provider
7. **Advanced Analytics:** Calculate trending topics across multiple articles

Troubleshooting

Problem: "Missing required configuration"

Solution: Check that all API keys are set in `.env` file

Problem: Rate limit errors **Solution:** Reduce number of articles or increase wait times in rate limiters

Problem: Tests failing **Solution:** Make sure all dependencies are installed: `pip install -r requirements.txt`

Problem: News API returns no articles **Solution:** Check your News API key, try different category or country

Problem: "Budget exceeded" error **Solution:** Increase `DAILY_BUDGET` in `.env` or process fewer articles